JavaScript Key Learnings — Question 4

OJT Assessment Prep — Optional Chaining, Nullish Coalescing & In Operator

---

1. Nested Objects

Objects can live inside other objects, creating a tree-like structure:

```
const booking = {
  id: 101,
  customer: {
    details: { name: "Alice" },
    contact: { phone: 7439878813 }
  }
};

// Access deep values by chaining dots
booking.customer.details.name  // → "Alice"
```

■■ **The problem:** if any middle layer is missing, JS throws a **TypeError** and crashes!

```
booking.customer.contact.phone
// if "contact" doesnt exist → TypeError: Cannot read properties of undefined
```

---

2. Optional Chaining (?.) — Safe Deep Access

?. means: if this exists, go deeper — if not, return **undefined** instead of crashing.

■ Without optional chaining — crashes if any level is missing:

```
booking.customer.contact.phone   // TypeError if contact missing!
```

■ With optional chaining — safe at every level:

```
booking.customer?.contact?.phone
// "Does customer exist?" → yes
// "Does contact exist?"  → no → returns undefined safely, no crash
```

■ *Use **?.** at every level you are unsure about. Chain as many as needed.*

```
booking.customer?.contact?.phone?.countryCode  // safe at every step
```

---

3. Nullish Coalescing (??) — Smart Default Values

?? means: if the left side is **null or undefined**, use the right side as the default.

Expression	Result	Why
`null ?? "default"`	"default"	left is null
`undefined ?? "default"`	"default"	left is undefined
`"hello" ?? "default"`	"hello"	left has a real value
`0 ?? "default"`	0	0 is not null/undefined

`"" ?? "default"`	`""`	empty string is not null/undefined

■■ ?? vs || — important difference:

```
0 || "default"   // → "default"  (|| treats 0 as falsy)
0 ?? "default"   // → 0          (?? only cares about null/undefined)
```

*Use **??** when 0 or empty string are valid values you want to keep.*

---

4. Combining ?. and ?? — The Power Pattern

These two operators are designed to work together for safe deep access with a fallback:

```
const phone = booking.customer?.contact?.phone ?? "No Phone Provided";
//                            ↑ safe access        ↑ fallback if anything missing

// If phone exists   → use the phone number
// If anything missing → undefined → "No Phone Provided"
```

■ *This is the standard pattern used in real production code for handling unreliable API data.*

---

5. The in Operator — Check if Property Exists

in checks if a **key name** exists in an object. Returns **true** or **false**.

```
const obj = { name: "Alice", age: 21 };

"name" in obj          // → true  (key exists)
"discountCode" in obj  // → false (key missing)
"age" in obj           // → true
21 in obj              // → false (21 is a value, not a key!)
```

■■ Important: in checks for the key (as a string), not the value.

```
// For this question:
let hasDiscount = "discountCode" in booking;
// → true if discountCode key exists anywhere in booking
// → false if it does not exist
```

---

6. Dynamic DOM Creation (Recap)

Same 3-step pattern — create, set content, append:

```
let p = document.createElement("p");     // Step 1 — create
p.textContent = `Phone: ${phNum} | Discount Applied: ${code}`; // Step 2 — content
div.appendChild(p);                       // Step 3 — append
```

■■ Full Correct Flow

```
const booking = { id: 101, customer: { details: { name: "Alice" } } };

// 1. Get the container div
let div = document.getElementById("summary-card");
```

```
// 2. Safe deep access + fallback
let phNum = booking?.customer?.contact?.phone ?? "No Phone Provided";

// 3. Check property existence
let hasDiscount = "discountCode" in booking;

// 4. Create and append DOM element
let p = document.createElement("p");
p.textContent = `Phone: ${phNum} | Discount Applied: ${hasDiscount}`;
div.appendChild(p);
```

■ Quick Reference — Modern JS Operators

Operator	Symbol	Use it when...
Optional Chaining	`?.`	Accessing nested properties that might not exist
Nullish Coalescing	`??`	Providing a default for null/undefined values
in Operator	`in`	Checking if a key exists in an object
Logical OR fallback	`||`	Providing a default for any falsy value (0, "", false)
